

VITreeM - Um Algoritmo baseado em Árvores para Alocação de Infraestruturas Virtuais

Ramon de Oliveira, Guilherme Piegas Koslovski

¹Programa de Pós Graduação em Computação Aplicada (PPGCA)
Universidade do Estado de Santa Catarina - UDESC - Joinville, SC - Brasil

ramon.oliveira@joinville.udesc.br, guilherme.koslovski@udesc.br

Resumo. As nuvens computacionais introduziram o provisionamento dinâmico de Infraestruturas Virtuais (IVs) disponibilizando computação, armazenamento e comunicação como serviços. Uma das questões investigadas nesse cenário é a alocação eficiente de recursos físicos para hospedar IVs. Por se tratar de um problema NP-Difícil, a busca por soluções ótimas possui um limite de aplicabilidade em cenários reais. Este artigo propõe um algoritmo para realizar essa tarefa, aplicando heurísticas baseadas em árvores para reduzir o espaço de busca. A proposta compreende uma estratégia para transformar grafos físicos e virtuais em árvores, que posteriormente, são simplificadas através da contabilização agrupada de informações. Essas inovações reduzem o número de comparações necessárias para identificar uma solução de alocação. Experimentos indicam que o algoritmo proposto encontra soluções em tempo viável para diversos cenários, mantendo uma alta taxa de aceitação e induzindo uma baixa fragmentação nos recursos físicos.

Abstract. Cloud Computing introduced dynamic provisioning of Virtual Infrastructures (VIs) delivering computing, storage and communication as services. One of the challenges investigated in this scenario is the efficient allocation of physical resources to host VIs. Due to its NP-Hard nature, the search for optimal solutions has a limited applicability in real scenarios. This paper proposes an algorithm to accomplish this task applying tree-based heuristics to reduce the search space. The proposal includes a strategy to transform physical and virtual graphs into trees, which subsequently are simplified by grouped accounting information. These innovations reduce the number of required comparisons to allocate a VI. Experiments indicate that the proposed algorithm finds solutions in feasible time for different scenarios while maintaining a high acceptance rate and the low physical infrastructure fragmentation.

1. Introdução

A computação em nuvem introduziu a entrega de recursos (armazenamento, comunicação e processamento) virtualizados sob demanda, abstraindo a preocupação, por parte do usuário, sobre o gerenciamento, manutenção e localização dos recursos físicos [Zhang et al. 2010] [Mell and Grance 2011]. Especificamente, os provedores de *IaaS* (*Infrastructure as a Service*) ofertam máquinas virtuais (MVs), roteadores e redes virtuais como serviço, compondo as Infraestruturas Virtuais (IVs). Para que as IVs possam ser provisionadas, o provedor deve alocá-las ¹ sobre os recursos físicos, isto

¹Seguindo a literatura, os termos mapeamento e alocação possuem o mesmo significado neste artigo.

é, além de alocar as MVs é necessário mapear os roteadores e canais que compõem a IV [Fischer et al. 2013]. Formalmente, IVs e recursos físicos, são representados como grafos: os vértices denotam os roteadores e as MVs enquanto as arestas representam os canais de comunicação. Sob esse ângulo, o provedor deve encontrar um mapeamento do grafo da IV sobre o grafo físico. Essa tarefa pertence ao conjunto de problemas NP-Difícil, pois pode ser reduzida a outros cenários já provados serem deste conjunto [Chowdhury and Boutaba 2010].

A literatura especializada compreende trabalhos que exploram o problema de alocação de recursos buscando soluções ótimas ou aplicando heurísticas [Yu et al. 2008], [Chowdhury et al. 2009], [Cheng et al. 2011], [Alkmin et al. 2013], [Wang et al. 2013], [Cavalcanti et al. 2014]. Embora eficientes em seus domínios de aplicação, constata-se que os trabalhos, em sua maioria, solucionam apenas a alocação de redes virtuais (*VNE - Virtual Network Embedding*), ou seja, roteadores e canais de comunicação, não considerando a existência das MVs e, consequentemente, limitando o número de recursos participantes da formulação do problema. Ademais, a busca por soluções ótimas possui uma aplicabilidade limitada em diversos cenários. Como observado em [Luizelli et al. 2013], as propostas nem sempre utilizam topologias que representam cenários reais (físicos ou virtuais), afirmação que pode influenciar na taxa de aceitação das requisições.

Nesse contexto, este trabalho propõe um algoritmo heurístico possuindo como premissa a definição de que ambos os grafos (IV e físico) são representados na forma de árvores, ou seja, os grafos devem ser conexos e acíclicos. Essa premissa permite a diminuição do espaço de busca através da aplicação de estratégias para abstração das subárvores. Para permitir o uso do algoritmo com qualquer topologia física e virtual, é apresentada uma estratégia para transformação de grafos em árvores. Os resultados obtidos são avaliados considerando métricas como taxa de aceitação, tempo de mapeamento, carga e fragmentação do *datacenter*. Em suma, as principais contribuições deste trabalho são: i) a definição de um algoritmo para transformação de topologias (física ou IVs) em árvores; ii) um algoritmo para abstrair e simplificar a representação das capacidades requisitadas e residuais; iii) um algoritmo para alocação de IVs baseado em árvores.

O restante deste trabalho está organizado da seguinte forma: a Seção 2 formaliza o problema de alocação de IVs enquanto a Seção 3 revisa os trabalhos relacionados. A Seção 4 descreve a solução proposta. A análise experimental é discutida na Seção 5. A Seção 6 apresenta as considerações finais e perspectivas para trabalhos futuros.

2. Alocação de Infraestruturas Virtuais

Através da virtualização, provedores IaaS podem abstrair os recursos disponíveis em *datacenters* para hospedar entidades virtuais isoladas, pertencentes a usuários distintos. Por exemplo, um único nó físico pode hospedar diversos nós virtuais (o mapeamento de nós virtuais para nós físicos é uma relação de $n:1$), enquanto canais virtuais podem se estender por vários canais da infraestrutura física (um caminho) [Chowdhury and Boutaba 2010]. Assim, um canal virtual que conecta dois nós virtuais u e v é mapeado sobre o caminho que conecta os nós físicos que hospedam os nós virtuais u e v (o mapeamento de canais virtuais para caminhos físicos é visto como um relacionamento $n:m$) [Fischer et al. 2013].

Em cenários típicos, existem restrições que devem ser respeitadas durante o mapeamento. Ou seja, a infraestrutura física deve ser capaz de suportar as capacidades soli-

citadas para os recursos virtuais. Por exemplo, os nós físicos devem suportar os requisitos mínimos de capacidade de processamento, memória e armazenamento exigidos pelo nó virtual. De forma semelhante, os canais físicos de comunicação devem suportar os requisitos de largura de banda, nível de confiança e qualidade de serviço exigidos pelo canal virtual [Alkmin et al. 2013] [Cavalcanti et al. 2014]. É sabido que a alocação não pode ser realizada aleatoriamente pois existem objetivos a serem alcançados. Ainda, a elasticidade de uma IV deve ser contemplada dinamicamente, ou seja, periodicamente um mecanismo deve verificar a necessidade de alteração dos recursos reservados. Usualmente, provedores buscam alocar o maior número de IVs de uma maneira eficiente, maximizando a taxa de aceitação e simultaneamente minimizando o número de recursos físicos compromissados. Ademais, as IVs podem ser posicionadas em qualquer lugar, porém o mecanismo de alocação deve reduzir o espalhamento e a fragmentação dos recursos físicos, diminuindo o número de recursos ativos e os custos administrativos.

Formalmente, o problema de alocar IVs sobre a infraestrutura física pode ser descrito como uma extensão da classe de problemas VNE [Yu et al. 2008] [Lischka and Karl 2009] [Chowdhury et al. 2009]: $G = (N, E, C)$ denota uma infraestrutura física, na qual N representa o conjunto de nós físicos (servidores e roteadores), E o conjunto de canais físicos, e cada nó ou canal físico $a \in N \cup E$ está associado com o vetor de capacidade $C(a)$ indicando a quantidade de recursos que a oferece. De forma semelhante, $G_v = (N_v, E_v, C_v)$ denota uma requisição de IV, onde N_v e E_v representam o conjunto de nós e canais virtuais, respectivamente, e C_v é o vetor de capacidades de um nó ou canal virtual. Assim, o mapeamento de IVs sobre os recursos físicos é descrito como $M : G_v \rightarrow (M_N, M_P)$, onde $M_N : n^i \in N_v \rightarrow n \in N$ é um mapeamento de nó virtual para nó físico e $M_P : e^i \in E_v \rightarrow P$ é o mapeamento de um canal virtual sobre um conjunto P de canais físicos. Cada G_v é alocada considerando o estado atual de G (grafo residual). Por grafo residual entende-se o grafo com as capacidades dos nós e arestas reduzidas, de acordo com as alocações realizadas sobre estes. $M(G_v)$ é considerado um mapeamento válido se, para todo nó ou canal virtual de G_v , seu respectivo mapeamento não excede as capacidades em G , ou seja, $\forall n^i \in N_v, C_v(n^i) \leq C(M_N(n^i))$ e $\forall e^i \in E_v, C_v(e^i) \leq \min(C(l), \forall l \in P)$.

3. Trabalhos Relacionados

A literatura atual compreende diversas soluções visando a alocação de recursos computacionais para hospedar entidades virtuais [Fischer et al. 2013]. As soluções existentes, em sua maioria, abordam uma subclasse do problema: *Virtual Network Embedding (VNE)*. Basicamente, VNE denota a alocação de um conjunto de roteadores virtuais (interconectados por enlaces virtuais) sobre topologias físicas [Chowdhury and Boutaba 2010].

Dentre as soluções revisadas, a abordagem de [Yu et al. 2008] introduziu a decomposição de um canal virtual sobre múltiplos caminhos físicos. Essa abordagem, combinada com uma reorganização periódica dos canais, alcançou um melhor aproveitamento da infraestrutura física. Inicialmente, a heurística buscava um posicionamento dos roteadores virtuais, e posteriormente, o mapeamento dos canais de comunicação. Embora o foco do trabalho não tenha compreendido a alocação de MVs, a métrica utilizada para identificação de candidatos potencialmente complexos era baseada no grau de conectividade do nó e serviu como base de comparação para o algoritmo proposto na Seção 4.

Por sua vez, [Lischka and Karl 2009] apresentou uma formulação baseada na identificação de grafos isomórficos, efetuando uma busca exaustiva pela presença de isomorfismo. Os resultados obtidos indicaram um elevado tempo de mapeamento (minutos). Em [Chowdhury et al. 2009] introduziu-se a alocação combinada dos canais de comunicação e roteadores virtuais, ou seja, ambos passaram a ser mapeados em um único passo pelo algoritmo. Essa inovação motivou a inclusão das MVs na formulação do problema e a concepção do algoritmo proposto no presente trabalho. Embora a diversidade das topologias virtuais não tenha sido o alvo dos trabalhos citados, os resultados obtidos motivaram o desenvolvimento da proposta de [Luizelli et al. 2013], que quantificou o impacto das topologias na taxa de aceitação das requisições. Seguindo essa linha, [Wang et al. 2013] propôs uma solução para topologias baseadas em árvores. Entretanto, a abordagem não considerava a alocação combinada de recursos de comunicação e processamento, limitando a representação de requisições e topologias físicas apenas na forma de árvores. Em contrapartida, [Cheng et al. 2011] e [Gong et al. 2014] não limitaram a formulação do problema para apenas uma topologia, explorando as particularidades de cada uma para identificação dos nós críticos (grau de conectividade).

Em [Alkmin et al. 2013], a formulação do VNE foi discutida considerando diversas informações para representação dos requisitos virtuais e capacidades dos recursos físicos. Recentemente, alguns trabalhos abordaram uma formulação que, além da alocação conjunta dos recursos de comunicação (roteadores e enlaces), introduziu a alocação de MVs [Cavalcanti et al. 2014] [Ruck et al. 2014]. Embora o presente trabalho tenha a alocação de IVs como alvo, alguns algoritmos e métricas oriundos de VNE foram considerados como diretrizes na formulação da solução: tempo de mapeamento aceitável, alta taxa de aceitação, baixa fragmentação da infraestrutura física, impacto das topologias de rede no processo de mapeamento e definição do nó de partida. Assim, a Seção 4 propõe uma abordagem para alocação de IVs aplicando conceitos referentes a árvores para diminuição do espaço de busca.

4. Uma Nova Abordagem para a Alocação de IVs

Este artigo propõe uma solução heurística (VITreeM - *Virtual Infrastructure Tree Mapping*) para a alocação *online* de IVs sobre uma infraestrutura física. VITreeM possui como premissa que ambos os grafos, de IV e infraestrutura física, estejam na forma de árvores. Em teoria dos grafos, uma árvore é um grafo não direcionado G que satisfaz as seguintes condições: *i*) ser conexo, ou seja, existe um caminho entre quaisquer dois vértices de G ; e *ii*) não conter ciclos, ou seja, existe apenas um único caminho entre quaisquer dois vértices de G . Essa premissa é motivada pela utilização de árvores na composição de topologias de *datacenters* [Cisco 2007] [Niranjan Mysore et al. 2009] [Kandula et al. 2009] e pela possibilidade de aplicação de algoritmos polinomiais para alocação. Entretanto, é sabido que as topologias de infraestruturas físicas e virtuais não necessariamente são especificadas como árvores, infringindo a condição de não existência de ciclos [Luizelli et al. 2013].

Dessa forma, VITreeM introduz uma estratégia para transformação de grafos em árvores abrangendo assim qualquer topologia. Em suma, a solução é decomposta em: *i*) transformação dos grafos em árvores conservando a representação das topologias originais; *ii*) definição dos nós de partida (as raízes das árvores) para alocação; *iii*) agrupamento em caixas para simplificar a representação dos recursos e diminuir o espaço de busca; *iv*) mapeamento das árvores guiado pelas definições da Seção 2.

4.1. Transformação de Grafos em Árvores

A estratégia para transformação de grafos em árvores é aplicada para as topologias físicas e virtuais, antes de efetuar o mapeamento. Como os grafos de entrada devem ser conexos, a infração nas propriedades de árvore ocorre na existência de ciclos entre os nós. Para remover os ciclos, a estratégia de transformação consiste em obter a árvore geradora máxima do grafo. Uma árvore geradora de um grafo $G = (N, E)$ é um subgrafo $T = (N^T, E^T)$ que possui as propriedades de uma árvore e contém todos nós de G , ou seja, $N^T = N$ e $E^T \subseteq E$. Em grafos ponderados a árvore geradora máxima de G é uma árvore geradora com o maior peso. Existe a possibilidade de haver mais de uma árvore geradora máxima, neste caso todas são equivalentes [Baase 2009]. Para obtenção da árvore geradora máxima de um grafo, utilizou-se uma adaptação do algoritmo de Kruskal, originalmente desenvolvido para obter a árvore geradora mínima [West et al. 2001]. Para tal, foi alterada a ordem de avaliação das arestas, substituindo a ordenação não-decrescente por uma ordenação não-crescente.

No grafo de uma infraestrutura física, as arestas que não estão presentes na árvore geradora máxima não serão consideradas no mapeamento. Porém, dentre todas árvores geradoras obtidas, a árvore geradora máxima é a que tem maiores chances de sucesso no mapeamento devido a maximização dos pesos nos canais de comunicação. A Figura 1 apresenta um exemplo de um grafo de infraestrutura física e a árvore geradora máxima da mesma. Nota-se que as arestas (d, b) , (d, c) foram removidas no processo de obtenção da árvore geradora máxima.

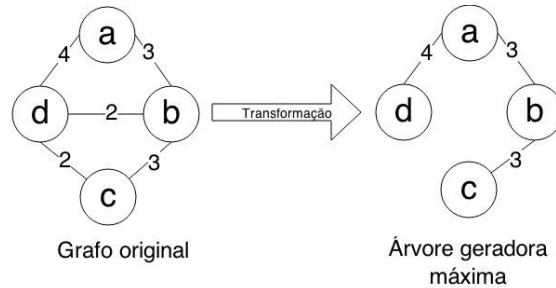


Figura 1. Definição de uma árvore geradora máxima para infraestruturas físicas.

No grafo de uma IV, $G_v = (N_v, E_v, C_v)$, descartar as arestas que não estão incluídas na árvore geradora máxima, $T_v = (N_v^T, E_v^T, C_v^T)$, não é uma opção aceitável, pois descaracteriza a estrutura da requisição. Para contornar este problema, o peso de toda aresta $(u, v) \in E_v - E_v^T$, é adicionado a toda aresta pertencente ao caminho entre u e v de T_v . A Figura 2 ilustra o processo de transformação aplicado a uma IV: a partir do grafo original obtém-se a árvore geradora máxima removendo as arestas (d, b) , $(d, c) \in E_v$. Por fim, adiciona-se o peso destas arestas na árvore geradora máxima entre os nós de origem e destino. Por exemplo, para a aresta (d, b) , seu caminho na árvore geradora máxima é $d-a-b$, logo, adiciona-se o peso da aresta $(d, b) \in E_v$ nas arestas (d, a) , $(a, b) \in E_v^T$. Assim, a aresta $(d, b) \in E_v$ passa a ser representada pelo caminho $d-a-b$ na árvore final, mantendo a conectividade entre os recursos virtuais conforme originalmente requisitado.

4.2. Nós de Partida para a Alocação

Por definição, o algoritmo precisa identificar um nó de cada árvore para iniciar a busca por uma solução. Conforme discutido em trabalhos anteriores [Yu et al. 2008]

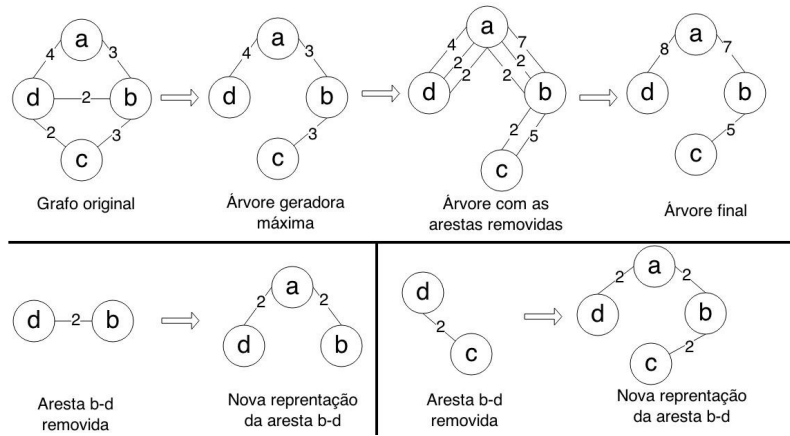


Figura 2. Processo de definição de uma árvore geradora máxima para uma IV.

[Gong et al. 2014], a definição do ponto de partida influencia na distribuição dos recursos virtuais sobre a infraestrutura física, pois ela define o modo como o algoritmo guiará a busca por recursos hospedeiros. Desta forma, a solução proposta utiliza duas estratégias para escolha dos nós iniciais: a identificação do centro da árvore e a busca pelo nó com a maior Capacidade Local de Recursos (*Local Resource Capacity* - LRC).

Em teoria dos grafos, a excentricidade de um nó n , pertencente a uma árvore, é definida como a maior distância do nó n para qualquer outro vértice. Por sua vez, o centro de uma árvore é um nó com a menor excentricidade e pode ser obtido através da remoção iterativa das folhas da árvore, restando apenas um ou dois nós [Knuth 1968]. A Figura 3(a) ilustra a obtenção do centro da árvore. As folhas (em vermelho) são removidas iterativamente até restar apenas o nó c que é o centro da árvore (em verde). A métrica LRC é utilizada para mensurar o potencial de mapeamento de um determinado nó, sendo calculada como o produto da capacidade do nó u e o somatório das capacidades das arestas diretamente conectadas ao nó ($LRC = C(u) * \sum C(u, v), \forall v \in N^T \wedge (u, v) \in E^T$) [Yu et al. 2008]. Quanto maior o valor do LRC para um nó, menor é a sua chance de ser mapeado devido a sua maior exigência de recursos. A Figura 3(b) ilustra o cálculo de LRC para cada um dos nós presentes na árvore, escolhendo o nó d com maior LRC.

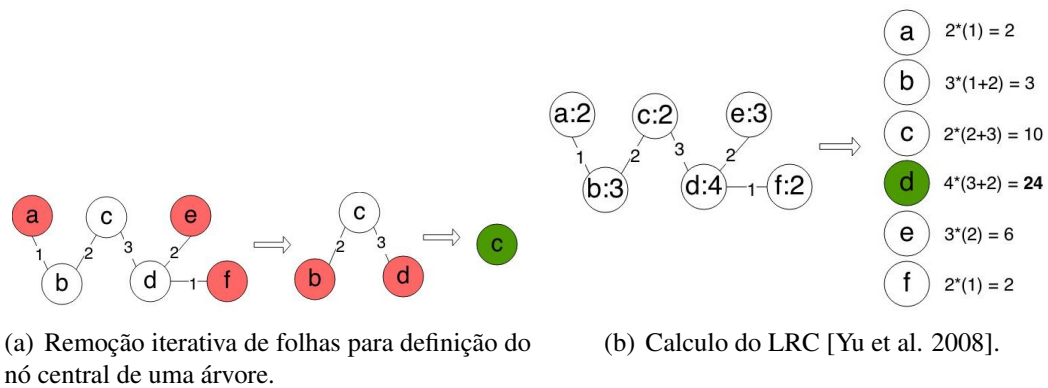


Figura 3. Identificação dos nós de partida para alocação.

4.3. Agrupamento dos Recursos em Caixas de Capacidades

Um dos principais desafios identificados na alocação de IVs é a explosão combinatória resultante do número de candidatos disponíveis em *datacenters*. É sabido que a abstração dos candidatos através do agrupamento das informações reduz o número de operações necessárias para verificar se um mapeamento é factível [Ricci et al. 2003]. VITreeM explora uma heurística, chamada de caixas, para agrupar as informações nos ramos de uma árvore, diminuindo o espaço de busca. Através das caixas de capacidades, o algoritmo possui uma noção dos recursos disponíveis a partir de um determinado nó, sem a necessidade de uma investigação exaustiva na árvore. Uma caixa contém uma tupla com três valores de capacidades (canais de comunicação, processamento e roteamento) obtidos através do somatório das capacidades nas subárvores formadas a partir de um determinado nó (os valores de uma caixa são alterados dependendo da referência utilizada).

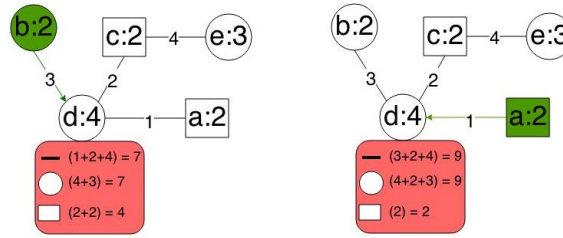


Figura 4. Exemplo de agrupamento de capacidades em caixas.

Na Figura 4 observa-se um exemplo do agrupamento em caixas de capacidades (em vermelho). Roteadores, servidores e canais de comunicação são representados por círculos, retângulos e linhas contínuas, respectivamente. A caixa do nó d , quando observada a partir do nó b ($caixa(b \rightarrow d)$) contém valores diferentes da caixa do nó d observada a partir do nó a ($caixa(a \rightarrow d)$), pois formam subárvores diferentes. O algoritmo utiliza essa abstração para ter uma noção da quantidade de recursos que estão presentes em uma subárvore, permitindo quantificar o seu potencial de mapeamento, sem a necessidade de explorar individualmente os elementos do espaço de busca.

4.4. Alocação de Infraestruturas Virtuais

O algoritmo VITreeM é o resultado da combinação das heurísticas anteriormente descritas, realizando as etapas na ordem em que foram apresentadas. Com foco na alocação *online* de requisições, este algoritmo recursivo aloca IVs individualmente em um determinado instante de tempo. O pseudocódigo apresentado no Algoritmo 1 descreve o funcionamento de VITreeM, recebendo como entrada: *i*) um grafo $G = (N, E, C)$ representando a infraestrutura física; *ii*) um grafo $G_v = (N_v, E_v, C_v)$ denotando a IV; *iii*) um nó central n_G da árvore T , sendo que $T = grafo_para_arvore(G)$; *iv*) o nó central n^v para a árvore T_v , sendo que $T_v = grafo_para_arvore(G_v)$; *v*) a árvore T ; *vi*) a árvore T_v e *vii*) e um mapeamento $M = (M_N, M_P)$ inicialmente vazio. Se alguma solução for encontrada, o algoritmo fornece como resultado um mapeamento M de G_v sobre G indicando quais recursos físicos foram utilizados para hospedar a requisição de IV.

Algoritmo 1: *VITreeM* - Alocação de IVs baseada em árvores.

Data: $G = (N, E, C)$; $G_v = (N_v, E_v, C_v)$; $n_G \in T$; $n^v \in T_v$; T ; T_v ;
 $M = (M_N, M_P)$;
Result: mapeamento M de G_v sobre G , caso possível

```
1 for  $\forall (u, n_G) \in T$  tal que  $u \in T$  do
2   if  $caixa(n_G \rightarrow u) \geq caixa(anterior(n^v) \rightarrow n^v)$  then
3     if  $VITreeM(G, G_v, u, n^v, T, T_v, M)$  then
4       return (verdadeiro,  $M$ );
5 if  $C(n_G) < C_v(n^v)$  then
6   return (falso, null);
7  $M_N(n^v) = n_G$ ;
8  $C(n_G) = C(n_G) - C_v(n^v)$ ;
9  $candidatos = \{(u \in N, v \in N_v), \forall (n_G, u) \in E \wedge (n^v, v) \in E_v\}$ ;
10 while  $\exists v \in N_v \wedge v \notin M_N \wedge \#candidatos > 0$  do
11    $(u, v) = proximo\_par(candidatos)$ ;
12    $candidatos = candidatos - \{(u, v)\}$ ;
13   if  $C((n_G, u)) \geq C_v((n^v, v)) \wedge caixa(n_G \rightarrow u) \geq caixa(n^v \rightarrow v)$  then
14      $C((n_G, u)) = C((n_G, u)) - C_v((n^v, v))$ ;
15      $M_P((n^v, v)) = M_P((n^v, v)) \cup (n_G, u)$ ;
16     if not  $VITreeM(G, G_v, u, v, T, T_v, M)$  then
17        $C((n_G, u)) = C((n_G, u)) + C_v((n^v, v))$ ;
18        $M_P((n^v, v)) = M_P((n^v, v)) - (n_G, u)$ ;
19 if  $\forall (n^v, v) \in E_v \wedge v \in M_N$  then
20   return (verdadeiro,  $M$ );
21 else
22   remover todo o mapeamento realizado a partir deste ponto;
23   return (falso, null);
```

Inicialmente, o algoritmo define uma tentativa de mapeamento de T_v , enraizada pelo nó n^v , sobre alguma subárvore de T enraizada pelos nós vizinhos de n_G (linhas 1 a 4). Para cada subárvore enraizada pelos vizinhos u de n_G , compara-se os valores da caixa de n^v (capacidade de máquinas virtuais, roteadores e canais de comunicação) aos valores da caixa de u . A função *anterior*() indica o candidato anteriormente alocado ou o próprio nó quando o algoritmo é iniciado. Esta restrição é um dos pontos chave do algoritmo, pois diminui o espaço de busca do problema, evitando a verificação desnecessária em nós profundos, quando detecta-se que a subárvore de n^v não é comportada pela subárvore de u . Caso a restrição não seja infringida, o algoritmo é executado recursivamente dando continuidade ao mapeamento (linha 3).

Se um mapeamento válido não for encontrado durante o processo descrito, tenta-se o mapeamento do nó n^v sobre o nó n_G (linhas 5 a 8). Se for possível, realiza-se o mapeamento das subárvores enraizadas por n^v sobre as subárvores enraizadas pelos nós vizinhos de n_G (linhas 9 a 18). Para tal, cria-se um conjunto (identificado por *candidatos*) com todos os possíveis pares entre os vizinhos de n_G e os vizinhos de n^v . A ordem em que os pares são selecionados segue um critério pré-estabelecido, como por exemplo *best-fit* que ordena a lista de pares de forma não-decrescente tendo como métrica a capacidade residual resultante da diferença entre as caixas do recurso físico e as caixas do recurso vir-

tual, ou *worst-fit* que ordena de forma não-crescente [Ruck et al. 2014]. Ao final, verifica-se a existência de um mapeamento válido para os vizinhos de n^v ; caso contrário, toda alocação realizada a partir da chamada corrente do algoritmo é desfeita. Este algoritmo possui uma complexidade assintótica de $O(n^2 \log(n))$ para o melhor caso, onde é possível fazer o mapeamento na sua primeira tentativa. Já para o pior caso, sua complexidade é $O(2^n \log(n))$, onde ele irá verificar todas as possibilidades de mapeamento.

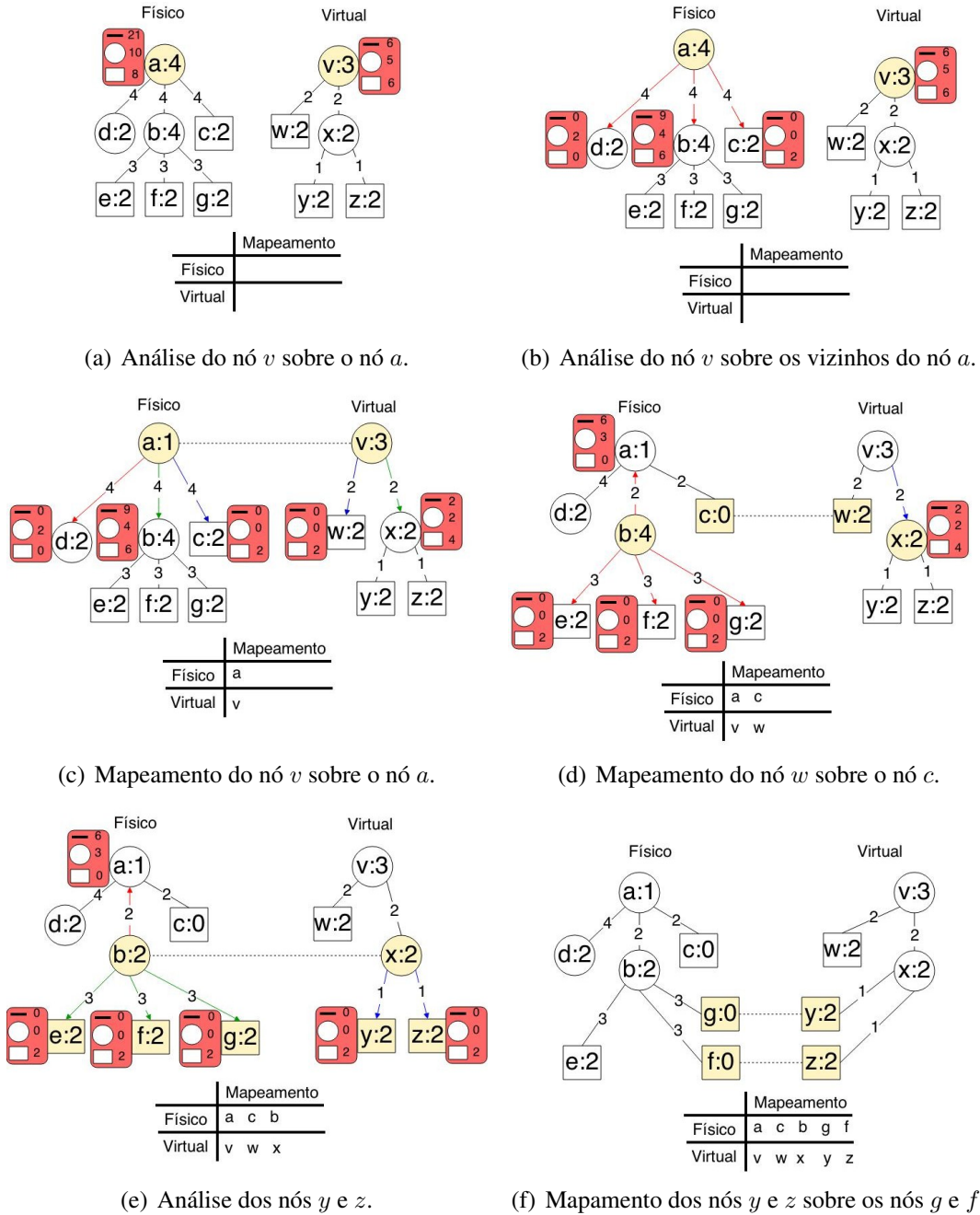


Figura 5. Exemplificação do algoritmo VITreeM.

A Figura 5 exemplifica o algoritmo em execução. Inicialmente, a Figura 5(a) descreve a análise do nó $a \in N^T$, da árvore física $T = (N^T, E^T, C^T)$, e do nó $v \in N_v^T$ da

árvore virtual $T_v = (N_v^T, E_v^T, C_v^T)$. Comparando os valores de cada item das duas caixas, observa-se a possibilidade de alocar a árvore T_v sobre a árvore T , pois os valores da caixa do nó a são maiores que os valores da *caixa* do nó v . O segundo passo (Figura 5(b)), busca um nó vizinho de a capaz de comportar a caixa do nó v (linhas de 1 a 4 do Alg. 1). Observa-se que não existe nenhum nó com recursos suficientes para comportá-lo, pois não há nenhuma caixa de um nó vizinho de a que comporte a caixa de v .

Como não foi possível avançar para os vizinhos de a , VITreeM realiza o mapeamento do nó v sobre o nó a (linhas 7 e 8 do Alg. 1), conforme ilustrado na Figura 5(c). Na sequência, o nó $w \in N_v^T$ é mapeado sobre o nó $c \in N^T$ pois não há vizinhos de c capazes de hospedar o nó w (Figura 5(d)). A mesma situação ocorre com o nó $x \in N_v^T$, não sendo possível avançar sobre nenhum vizinho de $b \in N^T$, pois estes não tem capacidades suficientes para comportá-lo. Devido a esta impossibilidade, x é mapeado sobre o nó b , restando assim apenas o mapeamento de seus vizinhos $y, z \in N_v^T$ (linhas 9 a 18 do Alg. 1). Como ilustrado pela Figura 5(e), apenas os nós $e, f, g \in N^T$ conseguem comportá-los, não havendo diferença na escolha dos mesmos, pois possuem capacidades iguais. A Figura 5(f) apresenta o estado final do mapeamento.

5. Análise Experimental

O algoritmo foi implementado em C++, compilado pelo *g++4.8* com o parâmetro de otimização *-O2*². Os testes foram executados em um equipamento com *AMD Phenom II X4*, 4 GB RAM e sistema operacional Ubuntu 14.04 (linux kernel 3.13).

5.1. Definição das Topologias Físicas e Virtuais

Seguindo o trabalho de [Cavalcanti et al. 2014], as infraestruturas físicas foram baseadas no modelo de três camadas da Cisco [Cisco 2007]. Assim, definiu-se que cada *datacenter* é composto por: i) camada de acesso, representada por 4 *racks*, cada um contendo 12 servidores e 1 *switch top-of-rack*, conectados por enlaces de 1 Gbps; ii) camada de agregação, com 4 *switches* interconectados por enlaces de 10 Gbps; iii) camada de núcleo, com 2 *switches* e enlaces de 10 Gbps. Para a simulação dos testes foram definidos três cenários baseados no *datacenter* descrito, que representam provedores com capacidades distintas. A Tabela 1 resume a quantidade total de nós para cada cenário.

Cenário	# <i>datacenters</i>	# servidores	# <i>switches</i>	# total
1	3	144	30	174
2	6	288	60	348
3	9	432	90	522

Tabela 1. Composição dos cenários de teste.

As requisições de IVs foram baseadas na topologia *fat-tree*, variando o número de camadas entre 2 e 7. Cada camada contém entre 2 e 5 *switches*, com exceção da última camada que possui entre 5 e 10 MVs. Seguindo trabalhos anteriores [Bays et al. 2012] [Oliveira et al. 2013] [Cavalcanti et al. 2014], cada MV, *switch* ou enlace virtual irá consumir entre 1 e 10% do recurso físico. Para cada cenário físico, foram geradas 100 requisições de IVs distribuídas entre 1 e 150 intervalos discretos (uma

²O código da implementação pode ser acessado em <https://bitbucket.org/ramoncs/VITreeM>

IV permanece ativa no máximo por 30 intervalos). Os valores foram distribuídos uniformemente.

5.2. Métricas para Análise

Foram selecionadas quatro métricas para análise de VITreeM: *i)* o *tempo médio de alocação* para uma requisição; *ii)* a *taxa de aceitação*, indicando o percentual de IVs mapeadas com sucesso, definida como a razão entre o número de IVs alocadas e o número de requisições; *iii)* a *fragmentação*, indicando a porcentagem de recursos ativos na infraestrutura física (que estão alocando elementos de alguma IV), definida pela divisão entre o número de recursos ativos e o total de recursos; *iv)* a *carga dos nós e dos canais físicos*, representando o percentual de recursos que estão sendo utilizados, ou seja, a divisão do somatório de capacidades alocadas pelo total existente, permitindo a visualização da capacidade total comprometida na alocação das IVs.

5.3. Análise dos Resultados

Para realizar uma análise comparativa, os resultados representam o comportamento de VITreeM perante a escolha do tipo de nó de partida (LRC ou centro) e da ordem em que os pares de candidatos são selecionados (*best-fit* ou *worst-fit*). Conforme discutido na Seção 4, a definição destes parâmetros afeta o desempenho dos algoritmos de alocação.

A Figura 6 apresenta os resultados para as métricas de taxa de aceitação (TA) e tempo de execução (TE). As Figuras 6(a), 6(c) e 6(e) resumem o tempo de mapeamento das requisições submetidas aos cenários 1, 2 e 3, respectivamente, bem como o desvio padrão para cada caso. Observa-se que VITreeM obteve um desempenho estável, com tempo médio de alocação abaixo de 3 segundos para todos os cenários, destacando-se as simulações que utilizaram *best-fit* como método de seleção de candidatos, pois apresentaram um desvio padrão menor que os demais. O tempo médio de alocação indica a possibilidade de utilização de VITreeM em cenários reais, já que usuários requisitantes aguardariam, em média, poucos segundos. A TA para os cenários 1, 2 e 3 (Figuras 6(b), 6(d) e 6(f)) indica que a utilização de VITreeM com a abordagem de ordenação de candidatos *best-fit* resulta em um maior número de requisições aceitas, independente da abordagem utilizada para definição do nó inicial. Especificamente, na Figura 6(f), as simulações com *best-fit* obtiveram aproximadamente 80% de TA enquanto *worst-fit* obteve aproximadamente 60%. Essa análise acompanha resultados anteriores [Ruck et al. 2014], ressaltando que, para as topologias analisadas, a utilização de LRC [Yu et al. 2008] pode ser substituída pela identificação do nó central sem prejuízos ao provedor hospedeiro.

A fragmentação da infraestrutura física (FG) e a carga dos canais de comunicação (CC) são apresentadas na Figura 7. Para FG (Figuras 7(a), 7(b) e 7(c)), observa-se nos cenários 2 e 3 uma maior fragmentação inicial quando o algoritmo *worst-fit* é aplicado. Após a estabilização (próximo dos intervalos 50 e 80 para os cenários 1 e 2, respectivamente), não se observam variações significativas. Analisando o cenário completo, a FG manteve-se estável, aproximadamente 20%, valor considerado baixo quando comparado a trabalhos anteriores [Cavalcanti et al. 2014]. Justifica-se a baixa FG, independente da abordagem de seleção de candidatos, pela natureza do algoritmo, que realiza o mapeamento guiado por uma busca em profundidade na árvore. O espalhamento (a busca em largura), usualmente ocorre apenas próximo das folhas.

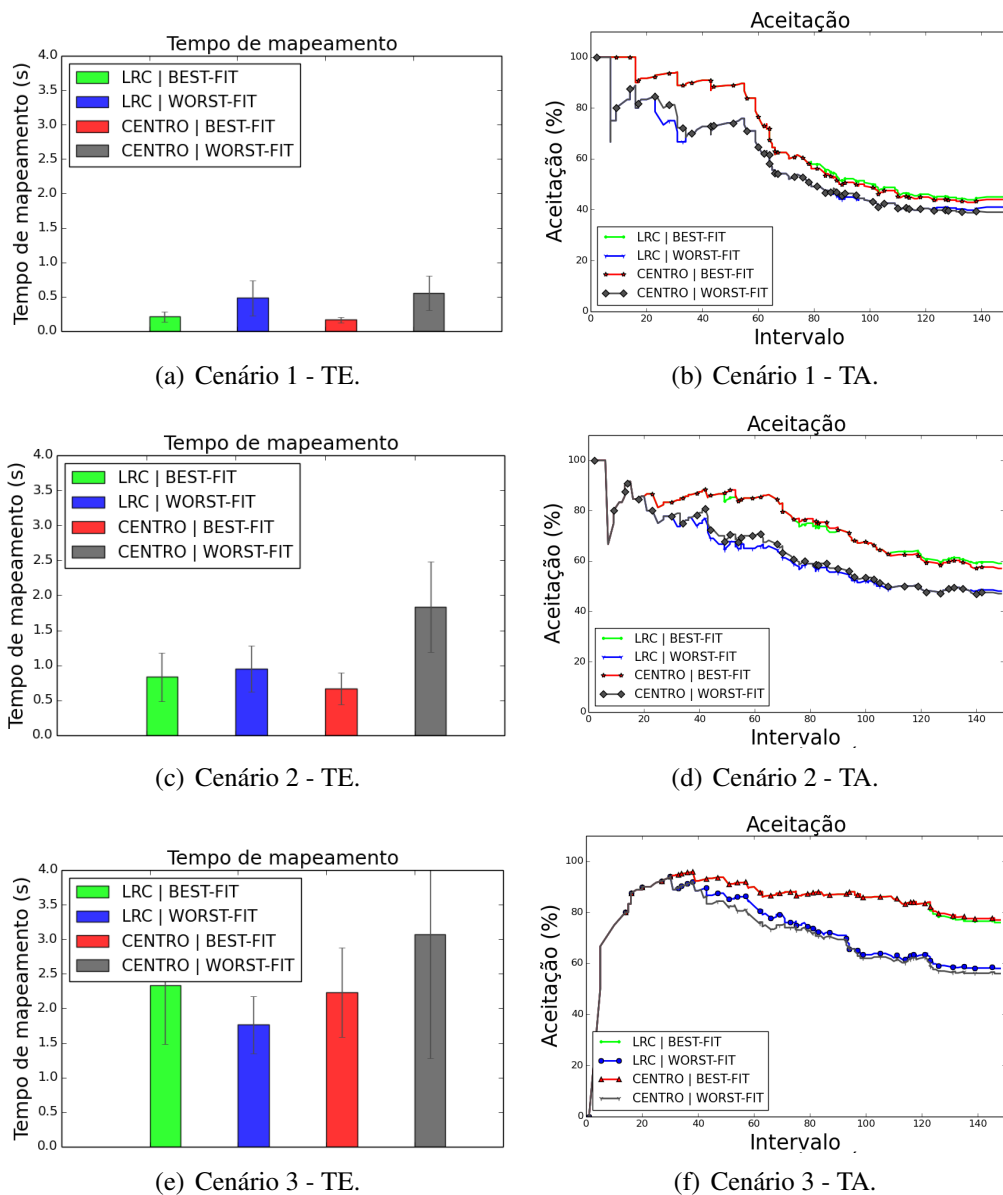


Figura 6. Resultados das simulações para as métricas de tempo de execução (TE) e taxa de aceitação (TA).

Quanto a carga computacional, as Figura 7(d), 7(e) e 7(f) apresentam a carga dos canais de comunicação (CC). Conforme observado na FG, as simulações que utilizaram *best-fit* para ordenação dos candidatos induziram uma menor CC no início das simulações. Porém, em regime estacionário, os valores estabilizaram em aproximadamente 25%, independente do cenário analisado. Observa-se que, embora VITreeM aproxime os recursos, a topologia física não é sobrecarregada. Por sua vez, os resultados para a métrica da carga dos nós (gráfico omitido) indicam que, em nenhum momento, o algoritmo saturou os recursos computacionais disponíveis. Em suma, os resultados obtidos mostram que a redução do espaço de busca introduzida por VITreeM permite sua aplicação em cenários reais. O algoritmo encontra soluções em tempo viável para diversos cenários, mantendo uma alta TA e induzindo uma baixa FG e CC.

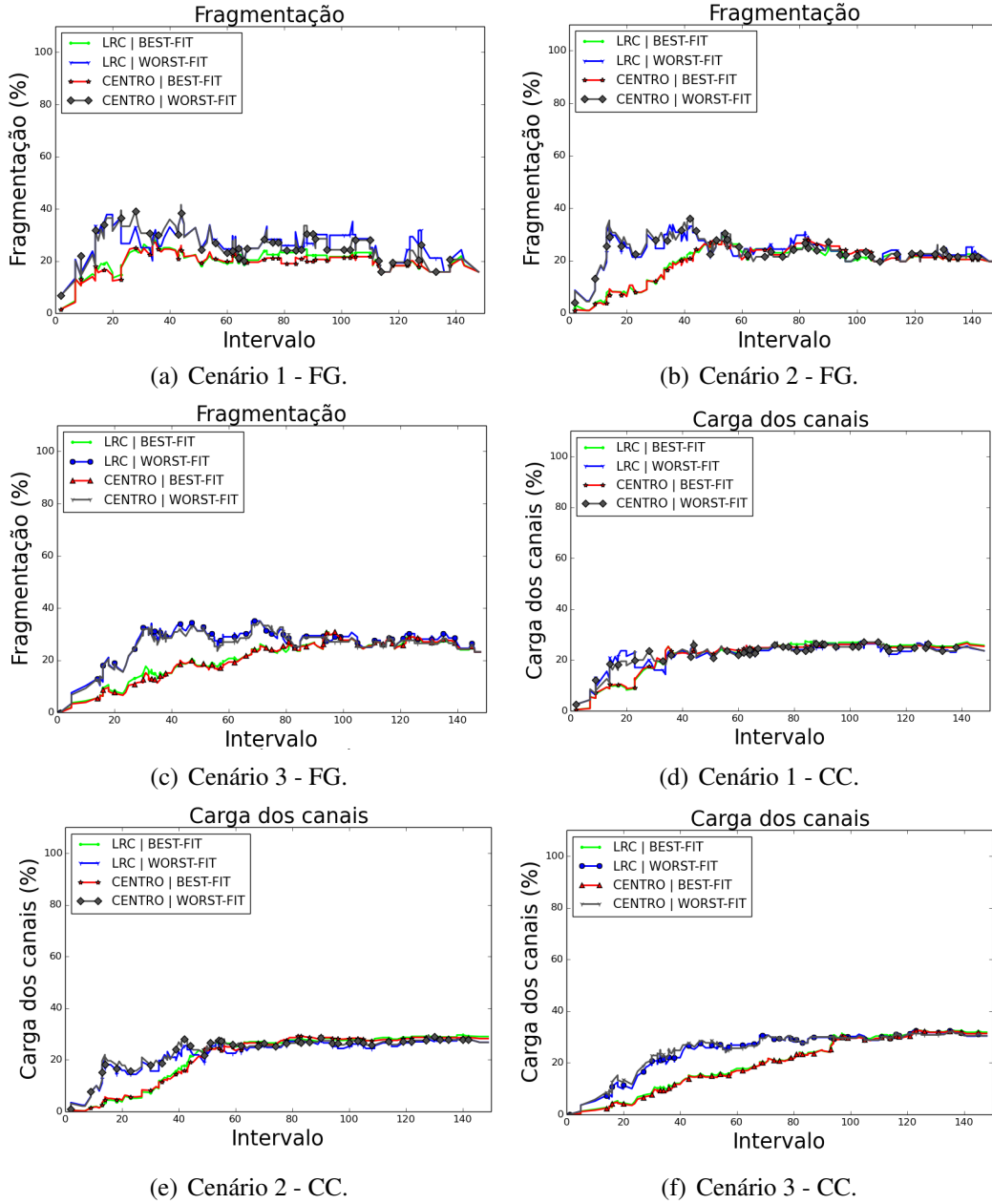


Figura 7. Resultados das simulações para as métricas de fragmentação (FG) e carga dos canais de comunicação (CC).

6. Conclusão

A computação em nuvem é um conceito terminantemente presente nos serviços que são disponibilizados através da Internet, destacando-se a entrega de IVs sob demanda. Dentre as tarefas de gerenciamento, a alocação de recursos físicos para hospedar as requisições de IVs é um problema pertencente a classe NP-Difícil. Nesse contexto, este artigo apresentou VITreeM, um algoritmo heurístico para alocação de IVs baseado em árvores. VITreeM transforma grafos (virtuais e físicos) em árvores e efetua a alocação restringindo o espaço de busca através do agrupamento de informações. A análise experimental indicou uma promissora utilização de VITreeM em diversos cenários, já que este manteve uma baixa

fragmentação do substrato combinada com uma alta taxa de aceitação. Para continuidade do trabalho consideraremos a elasticidade das IVs, o impacto da solução sobre aplicações n -camadas e a possibilidade de alocação de recursos em múltiplos provedores.

Referências

- Alkmin, G. P., Batista, D. M., and da Fonseca, N. L. S. (2013). Mapeamento de Redes Virtuais em Substratos de Rede. In *Proc. XXIX - SBRC*.
- Baase, S. (2009). *Computer algorithms: introduction to design and analysis*. Pearson Education India.
- Bays, L. R., Oliveira, R. R., Buriol, L. S., Barcellos, M. P., and Gaspary, L. P. (2012). Security-Aware Optimal Resource Allocation for Virtual Network Embedding. In *Proc. ACM/IEEE/IFIP CNSM*.
- Cavalcanti, G. A., Obelheiro, R. R., and Koslovski, G. P. (2014). Alocação ótima de recursos para infraestruturas virtuais confiáveis. In *Proc. XXXII - SBRC*.
- Cheng, X., Su, S., Zhang, Z., Wang, H., Yang, F., Luo, Y., and Wang, J. (2011). Virtual network embedding through topology-aware node ranking. *SIGCOMM Comput. Commun. Rev.*, 41(2):38–47.
- Chowdhury, N. and Boutaba, R. (2010). A survey of network virtualization. *Computer Networks*, 54(5):862–876.
- Chowdhury, N., Rahman, M., and Boutaba, R. (2009). Virtual network embedding with coordinated node and link mapping. In *INFOCOM 2009, IEEE*, pages 783–791.
- Cisco (2007). Cisco Data Center Infrastructure: 2.5 Design Guide.
- Fischer, A., Botero, J., Till Beck, M., de Meer, H., and Hesselbach, X. (2013). Virtual network embedding: A survey. *Communications Surveys Tutorials, IEEE*, 15(4).
- Gong, L., Wen, Y., Zhu, Z., and Lee, T. (2014). Toward profit-seeking virtual network embedding algorithm via global resource capacity. In *Proc INFOCOM. IEEE*.
- Kandula, S., Sengupta, S., Greenberg, A., Patel, P., and Chaiken, R. (2009). The nature of data center traffic: Measurements & analysis. In *Proc. of the 9th ACM SIGCOMM IMC*, New York, NY, USA.
- Knuth, D. E. (1968). The art of computer programming: Fundamental algorithms, vol. i.
- Lischka, J. and Karl, H. (2009). A virtual network mapping algorithm based on subgraph isomorphism detection. In *Proc. of the 1st ACM Workshop on Virtualized infrastructure systems and architectures*.
- Luizelli, M. C., Bays, L. R., Buriol, L. S., Barcellos, M. P., and Gaspary, L. P. (2013). Caracterizando o impacto de topologias no mapeamento de redes virtuais. In *Proc. XXXI - SBRC*.
- Mell, P. and Grance, T. (2011). The nist definition of cloud computing. In *Computer Security Division, Information Technology Laboratory, National Institute of Standards and Technology*.
- Niranjan Mysore, R., Pamboris, A., Farrington, N., Huang, N., Miri, P., Radhakrishnan, S., Subramanya, V., and Vahdat, A. (2009). Portland: a scalable fault-tolerant layer 2 data center network fabric. In *ACM SIGCOMM Computer Comm. Review*, volume 39.
- Oliveira, R. R., Bays, L. R., Marcon, D. S., Neves, M. C., Buriol, L. S., Gaspary, L. P., and Barcellos, M. P. (2013). DoS-Resilient Virtual Networks through Multipath Embedding and Opportunistic Recovery. In *Proc. ACM SAC*, pages 597–602.
- Ricci, R., Alfeld, C., and Lepreau, J. (2003). A solver for the network testbed mapping problem. *SIGCOMM Comput. Commun. Rev.*, 33(2):65–81.
- Ruck, D. B., Oliveira, R., and Koslovski, G. P. (2014). Comparação de algoritmos para alocação de Infraestruturas Virtuais. *Revista Brasileira de Computação Aplicada*.
- Wang, G., Zhao, Z., Lu, Z., Tong, Y., and Wen, X. (2013). A virtual network embedding algorithm based on mapping tree. In *13th Symp. Comm. and Information Technologies (ISCIT)*, pages 243–247.
- West, D. B. et al. (2001). *Introduction to graph theory*, volume 2. Prentice hall Upper Saddle River.
- Yu, M., Yi, Y., Rexford, J., and Chiang, M. (2008). Rethinking virtual network embedding: substrate support for path splitting and migration. *ACM SIGCOMM Computer Communication Review*.
- Zhang, Q., Cheng, L., and Boutaba, R. (2010). Cloud computing: state-of-the-art and research challenges. *Journal of internet services and applications*, 1(1):7–18.